Unit 4: Hashing Techniques and Analysis

Efficient Data Retrieval and Collision Handling

Ivan Temesvari
Distinguished Professor
Oakton College

Introduction

- What is hashing?
  - Definition: Mapping keys to array indices using a hash function
  - Goal: Achieve near-constant time (O(1)) for insertion, deletion, and search operations
- Why is hashing important?
  - Efficient data retrieval in various applications: databases, symbol tables, caches, etc.

Hash Tables

Structure:

- Array (buckets) to store key-value pairs
- Hash function to map keys to bucket indices

Operations:

- Insertion: Calculate hash, store key-value pair in the corresponding bucket
- Retrieval: Calculate hash, access the bucket, search for the key

Collisions

Definition:

- When two or more keys hash to the same bucket index

Why collisions happen:

- Limited number of buckets
- Imperfect hash functions

Need for collision resolution techniques

Collision Resolution: Linear Probing

- Mechanism:
  - If a collision occurs, search for the next available empty bucket in a linear sequence
- Advantages:
  - Simple to implement
- Disadvantages:
  - Primary clustering: Long chains of occupied buckets, increasing search time
  - Performance degrades as the table fills up

Collision Resolution: Quadratic Probing

Mechanism:

- If a collision occurs, probe buckets at increasing quadratic offsets (1^2, 2^2, 3^2, ...)

Advantages:

- Reduces primary clustering compared to linear probing

Disadvantages:

- Secondary clustering: Keys with the same initial hash value tend to cluster
- Still requires careful handling of table load factor

Collision Resolution: Double Hashing

Mechanism:

- Uses a second hash function to determine the probe sequence in case of collisions

Advantages:

- Further reduces clustering compared to linear and quadratic probing
- Better performance at higher load factors

Disadvantages:

- More complex to implement
- Requires a well-chosen second hash function

Extendible Hashing

- Motivation:
  - Dynamically handle growing datasets
  - Avoid expensive rehashing operations
- Structure:
  - Directory: Array of pointers to buckets
  - Buckets: Store key-value pairs
  - Hash function: Maps keys to a bit string (global depth)
- Operations:
  - Insertion: If bucket is full, split it and update directory
  - Retrieval: Use hash to find bucket, search for key

Hash Function Properties

Deterministic: Same input always produces the same output
Uniform: Distributes keys evenly across the buckets
Efficient: Computes hash values quickly

Analysis of Hashing

- Time complexity:
  - Best case: O(1) (no collisions)
  - Average case: O(1) with a good hash function and low load factor
  - Worst case: O(n) (all keys hash to the same bucket)
- Load factor:
  - Ratio of number of elements to the number of buckets
  - Impacts performance: Higher load factor increases the likelihood of collisions

Choosing the Right Hashing Technique

- **Factors to consider:**
  - Data distribution
  - Frequency of operations
  - Memory constraints
  - Performance requirements

Conclusion

- **Key takeaways:**
  - Hashing provides efficient data retrieval
  - Collision resolution is crucial for handling conflicts
  - Different techniques offer tradeoffs in performance and complexity
  - Choose the right technique based on your application's needs

Case Studies

- **Case Study 1: Databases**
- **Problem:** Efficiently store and retrieve large volumes of structured data.
- **Solution:** Hash-based indexing
  - Primary keys or frequently queried columns are hashed to create an index.
  - Allows for rapid lookup of records based on their key values.
- **Benefits:**
  - Significantly faster data retrieval compared to linear search.
  - Enables efficient handling of queries involving specific key values.

Case Studies

- **Case Study 2: Caches**
- **Problem:** Store frequently accessed data in a fast but limited-size memory.
- **Solution:** Hash tables for cache implementation
  - Data items are stored as key-value pairs, where the key is usually the memory address or a unique identifier.
  - Hashing allows for quick lookup of cached data.
- **Benefits:**
  - Improved application performance due to faster access to frequently used data.
  - Efficient management of cache space.

Case Studies

➨ **Case Study 3: Compilers**

- **Problem:** Track variables and their attributes (type, scope, memory location) during code analysis.

- **Solution:** Symbol tables using hash tables
  - Variable names are hashed to store their associated information efficiently.
  - Enables quick lookup of variable details during compilation.

- **Benefits:**
  - Faster compilation times.
  - Facilitates semantic analysis and error checking.

Hash Table Insertion

```
FUNCTION insert(table, key, value)
    index = HASH_FUNCTION(key) % table_size
    WHILE table[index] is not empty
        //Linear probing for collision resolution
        index = (index + 1) % table_size
    table[index] = (key, value)
```

Hash Table Retrieval

```
FUNCTION retrieve(table, key)
    index = HASH_FUNCTION(key) % table_size
    WHILE table[index] is not empty
        IF table[index].key EQUALS key
            RETURN table[index].value
        //Linear probing RETURN NOT_FOUND
        index = (index + 1) % table_size
```

Double Hashing

```
FUNCTION insert(table, key, value)
    index = HASH_FUNCTION1(key) % table_size
    //Ensure step_size is not 0
    step_size = HASH_FUNCTION2(key) % (table_size - 1) + 1
    WHILE table[index] is not empty
        index = (index + step_size) % table_size
    table[index] = (key, value)
```